Automated Voting System

Complete Code Documentation

React + Zustand + Material UI + Vite

Version 1.0.0 — April 2026

Table of Contents

1. Project Overview

1.1 Architecture

1.2 Technology Stack

2. Dependencies

2.1 Production Dependencies

2.2 Dev Dependencies

3. Project Structure

4. State Management (Zustand Stores)

4.1 Auth Store

4.2 Data Store

4.3 Theme Store

5. Application Entry Points

5.1 main.jsx

5.2 App.jsx

6. Layout Components

6.1 AppLayout

6.2 AppHeader

6.3 AppSidebar

7. Page Components

7.1 Login

7.2 Dashboard

7.3 Candidates

7.4 Voters

7.5 Voting

7.6 Results

8. Chart Components

8.1 PieChartCard

8.2 StatCard

8.3 Leaderboard

9. Styles

9.1 main.scss

9.2 _variables.scss

10. Data Model

10.1 Candidate Schema

10.2 Voter Schema

10.3 User Session

11. Routing

12. localStorage Persistence

1. Project Overview

The Automated Voting System is a single-page application (SPA) that provides a secure digital election platform. It supports two roles: **Admin** (manages candidates, voters, views results) and **Voter** (casts votes). All data is persisted in the browser's localStorage.

1.1 Architecture

- **Framework:** React 19 with functional components and hooks
- **State Management:** Zustand 5 with three separate stores (auth, data, theme)
- **UI Library:** Material UI 7 with custom purple/green theme
- **Routing:** React Router DOM v7 with nested routes and route guards
- **Charts:** Recharts 3 for data visualization
- **Build Tool:** Vite 8 with React plugin
- **Styling:** SCSS + MUI's sx prop system

1.2 Technology Stack

Layer	Technology	Version
UI Framework	React	19.2.4
State Management	Zustand	5.0.12
Component Library	Material UI	7.3.9
Routing	React Router DOM	7.13.2
Charts	Recharts	3.8.1
Data Grid	MUI X DataGrid	8.28.1
Build	Vite	8.0.1
Styling	Sass/SCSS	1.98.0

2. Dependencies

2.1 Production Dependencies

Package	Version	Purpose
`react`	^19.2.4	Core UI library for building component-based interfaces
`react-dom`	^19.2.4	React DOM renderer for web applications
`react-router-dom`	^7.13.2	Client-side routing with nested routes, route guards, and navigation
`zustand`	^5.0.12	Lightweight state management using hooks, replacing React Context
`@mui/material`	^7.3.9	Material Design React components (Button, Card, Dialog, AppBar, etc.)
`@mui/icons-material`	^7.3.9	Material Design icon set as React components
`@mui/x-data-grid`	^8.28.1	Advanced data table with sorting, pagination, and filtering
`@emotion/react`	^11.14.0	CSS-in-JS runtime required by MUI for styling
`@emotion/styled`	^11.14.1	Styled component API required by MUI
`recharts`	^3.8.1	Chart library built on D3 for bar, pie, line, and area charts
`sass`	^1.98.0	SCSS preprocessor for component and global styles

2.2 Dev Dependencies

Package	Version	Purpose
`vite`	^8.0.1	Fast build tool and dev server with HMR (Hot Module Replacement)
`@vitejs/plugin-react`	^6.0.1	Vite plugin for React JSX transform and fast refresh
`eslint`	^9.39.4	JavaScript/JSX linter for code quality
`@eslint/js`	^9.39.4	ESLint recommended rules for JavaScript

`eslint-plugin-react-hooks`	^7.0.1	ESLint rules enforcing React hooks rules of hooks
`eslint-plugin-react-refresh`	^0.5.2	ESLint rules for React Fast Refresh compatibility
`globals`	^17.4.0	Global variable definitions for ESLint
`@types/react`	^19.2.14	TypeScript type definitions for React (IDE support)
`@types/react-dom`	^19.2.3	TypeScript type definitions for React DOM (IDE support)

3. Project Structure

```
src/
├── main.jsx                        # App entry point
├── App.jsx                         # Root component with routing
├── assets/
│   └── hero.png                    # Hero image asset
├── stores/
│   ├── authStore.js                # Authentication & voting state (Zustand)
│   ├── dataStore.js                # Candidates & voters CRUD (Zustand)
│   └── themeStore.jsx              # Theme mode toggle (Zustand + MUI)
├── components/
│   ├── layout/
│   │   ├── AppLayout.jsx           # Main authenticated layout wrapper
│   │   ├── AppHeader.jsx           # Top navigation bar
│   │   └── AppSidebar.jsx          # Side navigation drawer
│   └── charts/
│       ├── PieChartCard.jsx         # Reusable pie chart card
│       ├── StatCard.jsx            # Statistic display card
│       └── Leaderboard.jsx         # Candidate ranking list
├── pages/
│   ├── Login.jsx                   # Login page (admin/voter)
│   ├── Dashboard.jsx               # Admin dashboard with charts
│   ├── Candidates.jsx              # Candidate CRUD management
│   ├── Voters.jsx                  # Voter registration management
│   ├── Voting.jsx                  # Voter ballot page
│   └── Results.jsx                 # Election results & analytics
└── styles/
    ├── main.scss                   # Global styles
    └── _variables.scss             # SCSS variables
```

4. State Management (Zustand Stores)

Zustand replaces React Context for state management. Three separate stores handle different domains, each with its own state, actions, and persistence strategy.

4.1 Auth Store

src/stores/authStore.js

Manages user authentication, session persistence, and per-position voting tracking.

State

Property	Type	Description
`currentUser`	Object or null	Logged-in user with name, voterId, role, pollingDistrict
`votedPositions`	Object	Map of voterId to an object of position: candidateId pairs

Actions

Action	Parameters	Description
`login`	`(user)`	Sets currentUser, saves to localStorage, loads voter's voted positions
`logout`	`()`	Clears currentUser and removes from localStorage
`markVoted`	`(voterId, position, candidateId)`	Records a vote in the votedPositions tracking map
`hasUserVotedForPosition`	`(voterId, position)`	Returns boolean indicating if voter voted for a specific position
`hasUserVoted`	`(voterId)`	Returns boolean indicating if voter voted for any position

Persistence

Uses `localStorage` key `voting_currentUser`. On login, loads voted positions from `voting_voters` to restore voting state.

4.2 Data Store

src/stores/dataStore.js

Manages all candidate and voter data with CRUD operations, voting logic, and authentication. This is the largest store.

State

Property	Type	Description
`candidates`	Array	List of candidate objects with id, name, party, position, category, photo, campaignMessage, votes
`voters`	Array	List of voter objects with id, voterId, name, email, password, pollingDistrict, status, idPhoto, hasVoted

Actions

Action	Parameters	Description
`addCandidate`	`(candidate)`	Adds a new candidate with auto-generated ID and 0 votes
`updateCandidate`	`(id, data)`	Updates candidate fields by ID
`deleteCandidate`	`(id)`	Removes candidate by ID
`getCandidateById`	`(id)`	Returns a single candidate by ID
`getCandidatesByPosition`	`(position)`	Returns candidates filtered by position
`addVoter`	`(voter)`	Registers a new voter with duplicate ID check
`updateVoter`	`(id, data)`	Updates voter fields by internal ID
`deleteVoter`	`(id)`	Removes voter by internal ID
`getVoterById`	`(id)`	Returns voter by internal ID
`getVoterByVoterId`	`(voterId)`	Returns voter by national voter ID

`castVote`	`(candidateId, voterId, position)`	Increments candidate votes, records position on voter
`getEnrichedVoters`	`()`	Returns voters with derived hasVoted flag based on all positions completed
`authenticateVoter`	`(voterId, password)`	Validates credentials against Active voters
`resetVoting`	`()`	Resets all vote counts and voter positions
`resetAll`	`()`	Restores all data to initial seed values

Persistence

Uses `localStorage` keys `voting_candidates` and `voting_voters`. Includes cross-tab synchronization via the `storage` event listener so changes in one tab reflect in others.

Seed Data

Initial data includes 5 candidates (3 for President, 2 for Senator) across RPF-Inkotanyi, PSD, and PL parties, and 5 voters across 5 districts (Kigali, Huye, Musanze, Rubavu, Muhanga). One voter is Inactive.

4.3 Theme Store

src/stores/themeStore.jsx

Manages light/dark mode toggle and wraps the app in MUI's ThemeProvider.

State

Property	Type	Description
`mode`	string	`'light'` or `'dark'`

Actions

Action	Description
`toggleTheme`	Switches between light and dark mode

ThemeProvider Component

Reads `mode` from the store and creates a MUI theme with purple primary palette (#7c3aed), custom component overrides (rounded buttons, cards, dialogs), and wraps children in `MUIThemeProvider` + `CssBaseline`.

5. Application Entry Points

5.1 main.jsx

src/main.jsx

The application entry point. Renders the root `<App>` component inside React 19's `StrictMode` using `createRoot`.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

5.2 App.jsx

src/App.jsx

The root component defining the application's routing structure. Wraps everything in `ThemeProvider` (from Zustand themeStore) for MUI theming, then sets up `BrowserRouter` with nested routes.

Routes

Path	Component	Layout	Access
`/`	LoginPage	None (full screen)	Public
`/dashboard`	DashboardPage	AppLayout	Admin only
`/candidates`	CandidatesPage	AppLayout	Admin only
`/voters`	VotersPage	AppLayout	Admin only
`/voting`	VotingPage	AppLayout	Voter only

`/results`	ResultsPage	AppLayout	All authenticated
`*`	Redirect to /	None	-

6. Layout Components

6.1 AppLayout

src/components/layout/AppLayout.jsx

Protected layout wrapper used by all authenticated routes. Checks `currentUser` from `useAuthStore` and redirects to login if not authenticated. Renders the sidebar, header, and `<Outlet>` for nested route content. Manages mobile sidebar toggle state.

6.2 AppHeader

src/components/layout/AppHeader.jsx

Fixed top navigation bar showing:

- Mobile menu toggle button (hamburger icon on small screens)
- Breadcrumb navigation with current page name
- Light/dark theme toggle switch using `useThemeStore`
- Current user name, role chip, and avatar (click to logout) using `useAuthStore`

Reads: `currentUser`, `logout` from authStore; `mode`, `toggleTheme` from themeStore.

6.3 AppSidebar

src/components/layout/AppSidebar.jsx

Side navigation drawer with:

- App logo and branding
- User profile section (avatar, name, role chip)
- Navigation items grouped by section (Management, Voting, Analytics)
- Role-based filtering: admin sees Dashboard/Candidates/Voters; voter sees Vote Now
- Theme toggle and logout button in footer
- Desktop: fixed sidebar. Mobile: slide-out drawer with overlay

Reads: `mode`, `toggleTheme` from themeStore; `currentUser`, `logout` from authStore.

7. Page Components

7.1 Login

src/pages/Login.jsx

Full-screen login page with green gradient background. Supports two login modes:

- **Admin:** Hardcoded credentials (ID: 112005, Password: Admin@123). Redirects to /dashboard.
- **Voter:** Validates against voter database via `authenticateVoter` from dataStore. Requires Active status. Redirects to /voting.

Includes form validation for required fields and numeric-only voter ID. Uses `useAuthStore.login` and `useDataStore.authenticateVoter`.

7.2 Dashboard

src/pages/Dashboard.jsx

Admin dashboard showing election overview with:

- 4 stat cards: Total Candidates, Registered Voters, Votes Cast, Voter Turnout %
- Pie chart: Vote distribution among candidates
- Bar chart: Votes by candidate
- Pie chart: Voter turnout (voted vs not voted)
- Leaderboard: Sorted candidate ranking with vote counts
- Line chart: District turnout comparison

Reads: `candidates` and `getEnrichedVoters()` from dataStore. Uses `useMemo` for computed data.

7.3 Candidates

src/pages/Candidates.jsx

Admin page for candidate management with MUI DataGrid table. Features:

- DataGrid with columns: Photo (avatar), Name, Party, Position, Category (chip), Votes, Actions
- Add/Edit dialog with form: name, party (select), position (select), category (select), photo upload (JPEG/PNG, max 2MB), campaign message
- View dialog showing full candidate profile

- Delete confirmation dialog

Uses: `addCandidate`, `updateCandidate`, `deleteCandidate` from dataStore.

7.4 Voters

src/pages/Voters.jsx

Admin page for voter registration management with MUI DataGrid. Features:

- DataGrid with columns: Photo, Voter ID, Name, Email, District, Status (Active/Inactive chip), Voted status, Actions
- Register/Edit dialog: voter ID (numeric, min 10 digits), name, email (validated), password (requires uppercase, number, special char, 8+ chars), district (select), status (select), ID photo upload
- View dialog with voter details
- Delete confirmation dialog

Uses: `getEnrichedVoters()`, `addVoter`, `updateVoter`, `deleteVoter` from dataStore.

7.5 Voting

src/pages/Voting.jsx

Voter-facing ballot page. Features:

- Candidates grouped by position (President, Senator, etc.)
- Each candidate displayed as a card with photo, name, party, category, campaign message
- View dialog for full candidate profile
- Vote confirmation dialog with warning about single-vote-per-position rule
- After voting, position shows "Voted" chip and cards are dimmed/disabled
- All-voted banner when every position is filled

Reads: `candidates`, `castVote` from dataStore; `currentUser`, `markVoted`, `hasUserVotedForPosition`, `votedPositions` from authStore.

7.6 Results

src/pages/Results.jsx

Election results and analytics page. Features:

- 4 summary stats: Total Votes, Voters, Turnout %, Candidates
- Pie chart: Vote distribution

- Pie chart: Voter turnout
- Bar chart: Vote count by candidate
- Area chart: Votes by party
- Leaderboard: Sorted candidate ranking
- Line chart: Votes by position

Reads: `candidates` and `getEnrichedVoters()` from dataStore.

8. Chart Components

8.1 PieChartCard

src/components/charts/PieChartCard.jsx

Reusable MUI Card wrapping a Recharts PieChart. Accepts props:

- `title` - Card header text
- `data` - Array of { name, value } objects
- `colors` - Array of hex color strings for pie segments
- `height` - Chart height in pixels
- `outerRadius` - Pie outer radius
- `showLegend` - Whether to show legend below chart
- `labelFormatter` - Custom label function for pie labels

8.2 StatCard

src/components/charts/StatCard.jsx

Compact statistic display card with icon and gradient background. Accepts props:

- `label` - Statistic name
- `value` - Statistic value (number or string)
- `icon` - MUI icon component
- `gradient` - CSS gradient string for card background

8.3 Leaderboard

src/components/charts/Leaderboard.jsx

Displays candidates sorted by vote count with rank, name, party, vote count, and progress bar. Accepts props:

- `candidates` - Pre-sorted candidate array
- `totalVotes` - Total vote count for percentage calculation

9. Styles

9.1 main.scss

src/styles/main.scss

Global stylesheet imported in App.jsx. Contains base resets, scrollbar styling, and global CSS classes. Imports the variables partial.

9.2 _variables.scss

src/styles/_variables.scss

SCSS variables partial defining reusable values (colors, spacing, breakpoints) used throughout the SCSS stylesheets.

10. Data Model

10.1 Candidate Schema

```
{
  id: Number,              // Auto-generated unique identifier
  name: String,            // Full name (e.g., "Alice Uwimana")
  party: String,           // Political party (e.g., "RPF-Inkotanyi")
  position: String,        // Running position (e.g., "President")
  category: String,        // "National" or "Regional"
  photo: String,           // URL to avatar image
  campaignMessage: String, // Campaign description text
  votes: Number            // Accumulated vote count
}
```

10.2 Voter Schema

```
{
  id: Number,              // Auto-generated unique identifier
  voterId: String,         // National ID number (numeric, min 10 digits)
  name: String,            // Full name
  email: String,           // Email address
  password: String,        // Account password
  pollingDistrict: String, // District assignment (e.g., "District A - Kigali")
  status: String,          // "Active" or "Inactive"
  idPhoto: String,         // URL to ID photo
  hasVoted: Boolean,       // Derived: true when all positions voted
  votedPositions: Object   // Map of position: candidateId pairs
}
```

10.3 User Session

```
{
  name: String,            // Display name
  voterId: String|Number,  // Voter ID or admin ID
  role: String,            // "admin" or "voter"
```

```
  pollingDistrict: String  // District or "HQ" for admin
}
```

11. Routing

React Router DOM v7 with `BrowserRouter` and nested routes. The `AppLayout` component acts as a layout route, rendering the sidebar and header around `<Outlet>` child routes.

Route Guard

`AppLayout` checks `useAuthStore.getState().currentUser`. If null, it renders `<Navigate to="/" replace />`, redirecting unauthenticated users to the login page.

Navigation Filtering

`AppSidebar` filters navigation items based on user role:

- `adminOnly: true` items (Dashboard, Candidates, Voters) shown only to admins
- `voterOnly: true` items (Vote Now) shown only to voters
- Items without flags (Results) shown to all authenticated users

12. localStorage Persistence

The application uses the browser's localStorage for data persistence. No backend server is required.

Key	Store	Description
`voting_currentUser`	authStore	JSON-serialized current user session object
`voting_candidates`	dataStore	JSON-serialized array of all candidate objects
`voting_voters`	dataStore	JSON-serialized array of all voter objects (includes votedPositions)

Persistence Flow

- **Read:** On store initialization, data is loaded from localStorage with seed data as fallback
- **Write:** Every mutation (add, update, delete, vote, reset) immediately persists to localStorage
- **Cross-tab sync:** The dataStore listens to the browser `storage` event to sync changes across tabs

Seed Data

When localStorage keys are empty, stores initialize with hardcoded seed data: 5 candidates (President and Senator positions) and 5 voters (4 Active, 1 Inactive) across 5 districts.

Reset Operations

- `resetVoting()` : Sets all candidate vote counts to 0 and clears all voter votedPositions
- `resetAll()` : Restores all data to initial seed values